ASSESSMENT: PART 2

Module Code: PDAN8412

SEPTEMBER 21, 2026

ROSEBANK COLLEGE
Student Number: ST10465074
Name: Mafalo Mmakgaha

Contents

1. Introduction and objective 2
2. Dataset and initial inspection 2
3. PySpark setup, schema and data quality 3
3.1 Spark code used for setup and quality audit 4
3.2 Spark missing-value audit code 5
3.3 Scope clarification 5
4. Target definition and class balance 5
5. Exploratory data analysis 7
5.1 Distribution of Amazon sales rank 7
5.2 Correlation analysis 8
5.3 Bestseller versus non-bestseller averages 9
5.4 Bestseller rate by genre 9
5.5 Bestseller rate by publisher type 10
6. Feature selection 11
6.1 Features retained for the scratch model 13
7. Model preparation and training 13
8. Initial and improved model results 13
9. Threshold optimisation 15
10. Final evaluation and confusion matrix 16
11. Feature weights 17
12. Discussion 17
13. Limitations and recommendations 18
14. Conclusion 18
References 18
Appendix A. Jupiter Notebook 19

1. Introduction and objective

The objective is to explore the book-sales data and develop a binary classifier that predicts whether a book is a bestseller. The notebook operationalises bestseller status using the book's recorded Amazon sales rank. Since sales rank directly determines the target label, it is excluded from the predictors to avoid target leakage. The report follows the order of the executed notebook and retains the terminology.

Logistic regression is a standard approach for binary classification, estimating the probability of an outcome through the logistic function (James et al., 2021). Class imbalance can make accuracy alone misleading; precision, recall and F1-score therefore provide complementary views of positive-class performance (Sokolova and Lapalme, 2009).

2. Dataset and initial inspection

The dataset is the CORGIS Publishers dataset, which contains information about Amazon e-book sales, genres, publishers, revenue, units sold, ratings, prices, sales rank and reviews. The CORGIS documentation states that the underlying collection covers approximately 54,000 titles and notes that it is not time-oriented; it represents a collection of books rather than a time series. The supplied publishers.csv used for this analysis contains 27,027 records.

Dataset source: CORGIS Publishers Dataset

The CSV file publishers.csv was read into pandas for initial inspection. The first five records display contain genre and seller information, daily average revenue/sales/units, publisher name and type, average rating, sale price, sales rank and total reviews.

Dataset property	**Notebook result**
Shape	(27027, 13)
Categorical/object fields	genre; sold by; publisher.name; publisher.type
Numeric fields	daily average.amazon revenue; daily average.author revenue; daily average.gross sales; daily average.publisher revenue; daily average.units sold; statistics.average rating; statistics.sale price; statistics.sales rank; statistics.total reviews
Missing values	0 in every listed column
Duplicate rows	0

All 27,027 records are complete for the 13 listed columns, and no duplicated rows were reported by the notebook. The descriptive statistics show substantial dispersion in sales and revenue measures.

For example, daily average units sold has a mean of 29.61 and maximum of 7,000, while sales rank ranges from 1 to 99,974. These ranges indicate right-skew and extreme observations in several commercial variables.

Variable	Mean	Std. dev.	Minimum	Median (50%)	Maximum
Amazon revenue (daily average)	37.978937	250.995881	0.004	9.576	12974.0
Author revenue (daily average)	41.465654	223.321357	0.004	10.152	9559.0
Gross sales (daily average)	150.572864	865.653698	0.020	41.860	47795.0
Publisher revenue (daily average)	71.128273	481.337817	0.000	16.776	28677.0
Units sold (daily average)	29.614053	157.739500	1	9	7000
Average rating	4.159787	0.995815	0	4.40	5.0
Sale price	6.777488	6.368486	0.01	5.12	141.52
Sales rank	38200.142857	29056.493296	1	32810	99974
Total reviews	136.099345	468.952999	0	37	9174

3. PySpark setup, schema and data quality

The notebook then creates a SparkSession, reads publishers.csv with headers and schema inference, and normalises column names to lowercase underscore-separated names. The Spark schema reports strings for categorical fields, doubles for revenue/rating/price fields and integers for units sold, sales rank and total reviews. The Spark missing-value aggregation reports zero nulls across all 13 fields. These are the actual Spark outputs embedded in cells 3–4 of the notebook (AssessmentPart2.ipynb, 2026).

3.1 Spark code used for setup and quality audit

```python
## PySpark setup and column-name cleaning
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, count, sum as spark_sum, when, round as spark_round,
    avg, min as spark_min, max as spark_max
)

spark = (
    SparkSession.builder
    .appName("AmazonBookAnalysis")
    .getOrCreate()
)

spark_df = spark.read.csv(
    "publishers.csv",
    header=True,
    inferSchema=True
)

for old_name in spark_df.columns:
    new_name = old_name.lower().replace(" ", "_").replace(".", "_")
    spark_df = spark_df.withColumnRenamed(old_name, new_name)

spark_df.printSchema()
spark_df.show(5, truncate=False)
```

The schema output printed by this code is summarized below:

Spark field	Inferred type	Nullable
genre	string	true
sold_by	string	true
daily_average_amazon_revenue	double	true
daily_average_author_revenue	double	true
daily_average_gross_sales	double	true
daily_average_publisher_revenue	double	true
daily_average_units_sold	integer	true
publisher_name	string	true
publisher_type	string	true
statistics_average_rating	double	true

statistics_sale_price	double	true
statistics_sales_rank	integer	true
statistics_total_reviews	integer	true

3.2 Spark missing-value audit code

```
#Spark missing values
missing_exprs = [
    spark_sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in spark_df.columns
]

spark_df.select(*missing_exprs).show()
```

Exact Spark output: every column has a missing-value count of 0. This agrees with the pandas null-count audit and supports proceeding without a missing-value imputation step.

3.3 Scope clarification

The notebook uses PySpark for session creation, CSV ingestion, schema inspection and the null-count aggregation. The subsequent exploratory summaries, feature selection, dummy encoding, train/test split, custom logistic-regression implementation, metrics and plots are executed with pandas, NumPy, matplotlib/seaborn, statsmodels and scikit-learn. This report therefore reproduces only the actual Spark code in the Spark-code appendix and does not relabel the later Python modelling code as Spark.

4. Target definition and class balance

The notebook defines BestSeller = 1 when statistics. Sales rank is less than or equal to 5,000 and 0 otherwise. Sales rank is used only to create the label and is excluded from the candidate predictors because including it would leak the target definition into the model.

Sales-rank cut-off	**Books at or below cut-off**	**Share of dataset**
≤ 100	99	0.37%
≤ 500	453	1.68%
≤ 1,000	854	3.16%
≤ 5,000	3,566	13.19%
≤ 10,000	6,014	22.25%

Target class	Count	Percentage
Not Best Seller (0)	23,461	86.81%
Best Seller (1)	3,566	13.19%

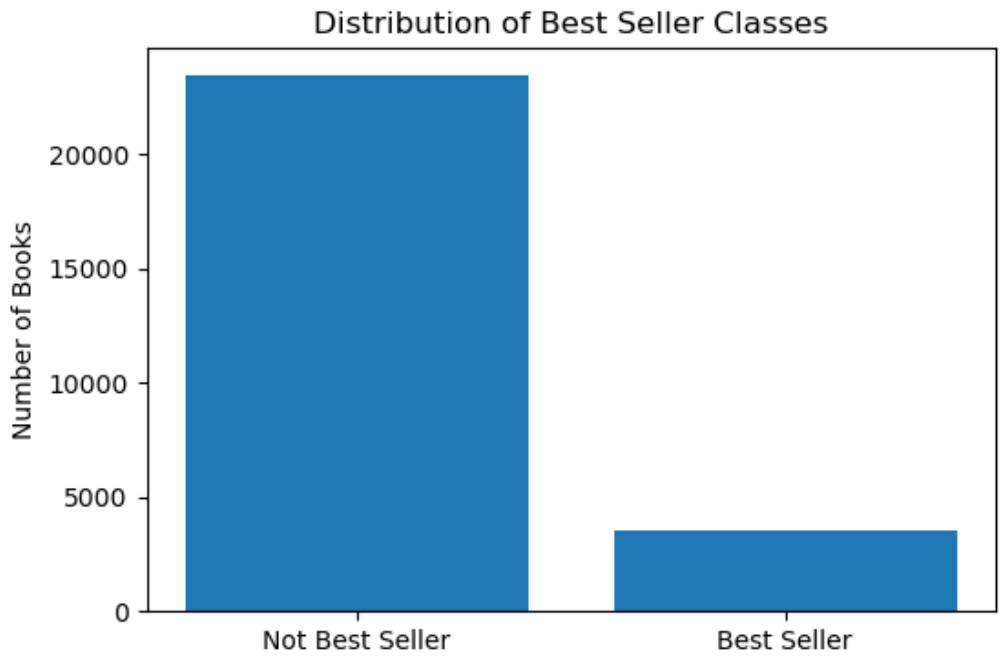


Figure 1. Distribution of Best Seller Classes.

The class distribution is imbalanced: fewer than one in seven records are labelled as bestsellers. This matters because a model can obtain high accuracy by predicting the majority class too often while missing positive cases. The report therefore considers positive-class recall and F1-score when comparing thresholds.

5. Exploratory data analysis

5.1 Distribution of Amazon sales rank

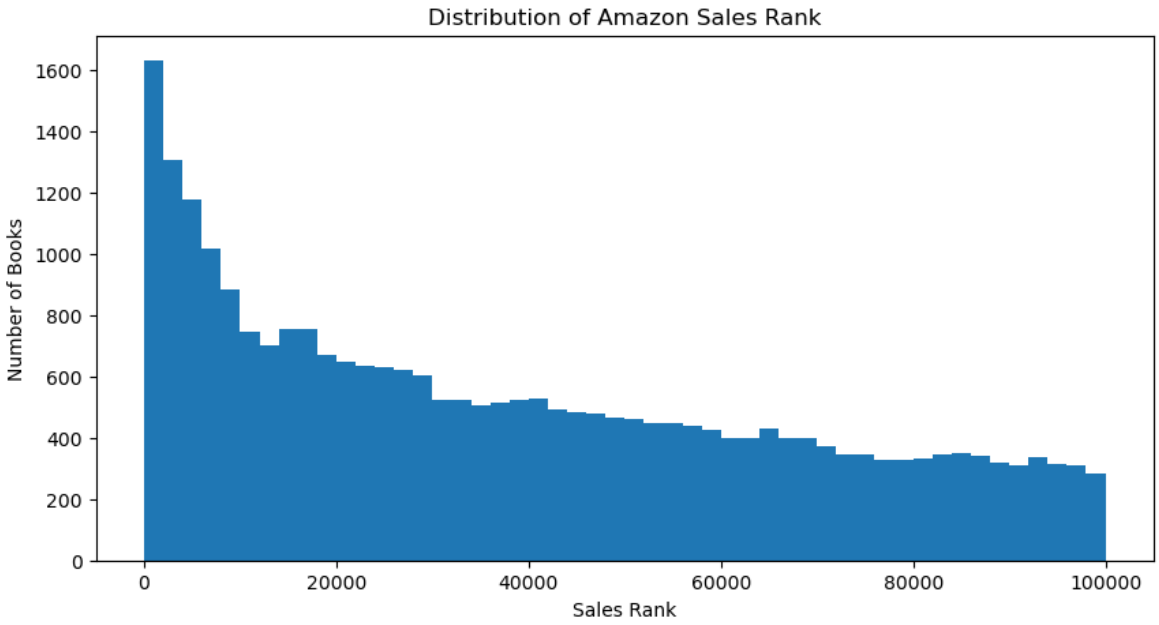


Figure 2. Distribution of Amazon Sales Rank across all records.

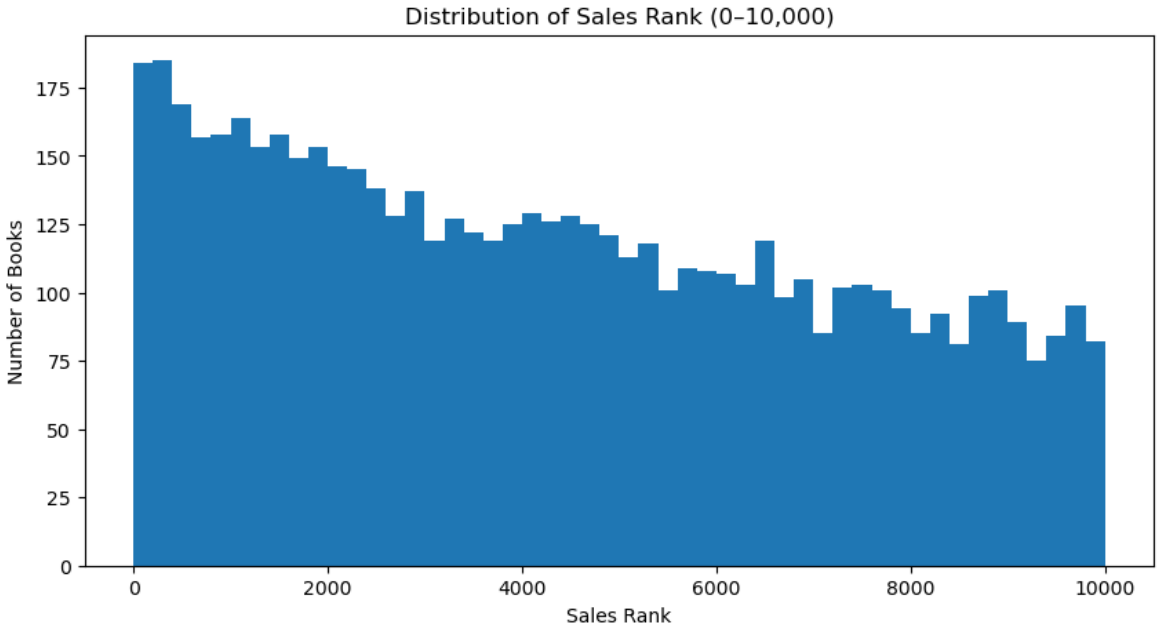


Figure 3. Distribution of Amazon Sales Rank for ranks 0–10,000.

The full-range and restricted-range histograms are included exactly as rendered in the notebook. The full range extends to a sales rank of 99,974, while the second figure zooms into ranks up to 10,000. The latter makes the dense low-rank region more visible, including the area used to define the positive class.

5.2 Correlation analysis



Figure 4. Correlation matrix for numerical variables and the bestseller indicator.

Variable correlated with BestSeller	Pearson correlation
statistics.sales rank	-0.481442
daily average.units sold	0.348056
daily average.author revenue	0.323436
statistics.total reviews	0.293715
daily average.gross sales	0.293240
daily average.amazon revenue	0.270493
daily average.publisher revenue	0.236261
statistics.sale price	-0.108989

statistics.average rating	0.007968

Sales rank has the strongest absolute correlation with the target, at −0.481442, which is expected because rank is used to define bestseller status. It is excluded from modelling. Among the remaining numeric measures, average units sold have the strongest positive correlation (0.348056), followed by author revenue and total reviews. Average rating has a near-zero linear correlation (0.007968), while sale price is weakly negative (−0.108989). Correlation describes pairwise linear association and should not be interpreted as causal effect.

5.3 Bestseller versus non-bestseller averages

Measure	Not Best Seller	Best Seller
Daily average Amazon revenue	11.510298	212.118233
Daily average author revenue	13.305937	226.730686
Daily average gross sales	51.609108	801.663348
Daily average publisher revenue	26.792873	362.814429
Daily average units sold	8.209880	170.433819
Average rating	4.156693	4.180137
Sale price	7.048089	4.997182
Total reviews	82.400580	489.387830

The group means showing markedly higher sales, revenue and review counts for bestseller-labelled books. Average units sold are about 170.43 for bestsellers versus 8.21 for non-bestsellers. Bestseller books also have a lower average sale price (4.997182 versus 7.048089), while mean ratings are very similar (4.180137 versus 4.156693). These are descriptive differences rather than causal estimates.

5.4 Bestseller rate by genre

Genre	Total books	Bestsellers	Bestseller percentage
fiction	733	248	33.83%
genre fiction	8,903	2,512	28.22%
nonfiction	14,161	715	5.05%
children	2,541	88	3.46%
comics	568	3	0.53%
foreign language	121	0	0.00%

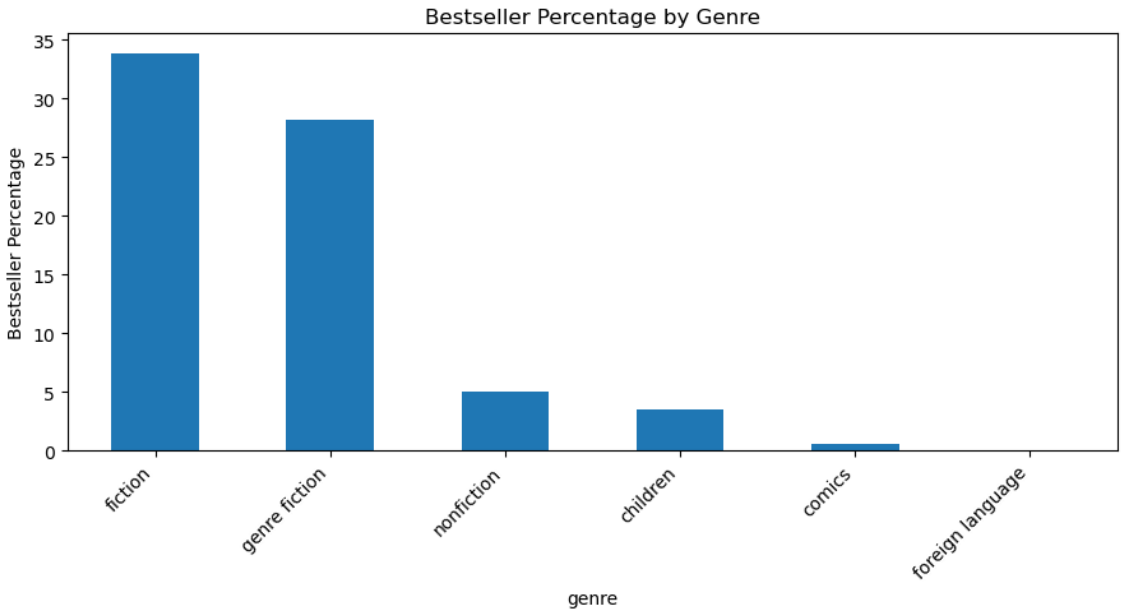


Figure 5. Bestseller Percentage by Genre.

The highest bestseller rate appears in the genre category “fiction” (33.83%), followed by “genre fiction” (28.22%). The rates are lower for nonfiction (5.05%) and children (3.46%), and very low for comics (0.53%); no bestseller is observed in the foreign-language group in this sample. The notebook retains “fiction” and “genre fiction” as separate source categories.

5.5 Bestseller rate by publisher type

Publisher type	Total books	Bestsellers	Bestseller percentage
amazon	423	187	44.21%
indie	5,946	1,147	19.29%
big five	7,309	1,333	18.24%
single author	3,608	323	8.95%
small/medium	9,741	576	5.91%

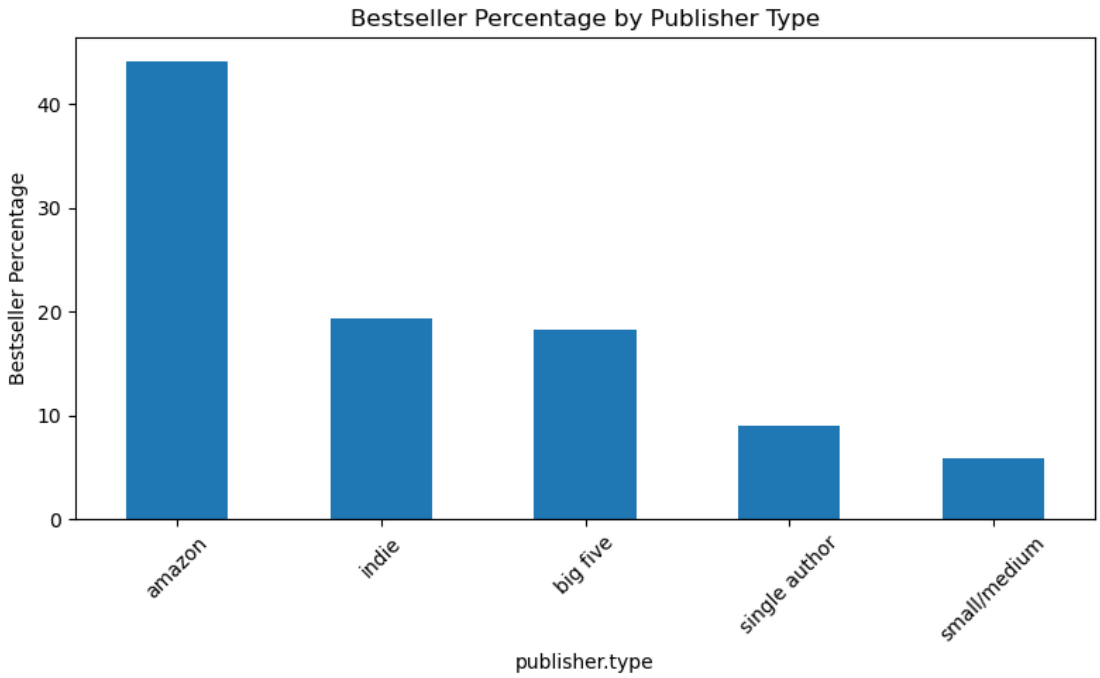


Figure 6. Bestseller Percentage by Publisher Type.

The “amazon” publisher-type category has the highest observed bestseller percentage (44.21%), though it contains 423 books, considerably fewer than the largest groups. Indie publishers and the big-five group have rates of 19.29% and 18.24%, respectively. The small/medium category has the lowest rate among the groups listed (5.91%). Group sizes differ, so rates should be interpreted together with counts.

6. Feature selection

Candidate predictors in the notebook are genre, publisher type, average rating, sale price and total reviews. The selection step uses statsmodels Logit on 26,906 observations after category handling; the notebook replaces the rare “foreign language” genre with “other” for this screening model and groups sellers with fewer than ten records. The printed regression output is reproduced below. This screening regression includes average rating even though it is not retained in the final scratch model.

Logit output item	**Value**
Dependent variable	BestSeller
Observations	26,906
Df Model / Df Residuals	11 / 26,894
Pseudo R-squared	0.2827
Log-Likelihood	-7549.6
LL-Null	-10525.
LLR p-value	0.000

Converged	True; 9 iterations

Term	Coefficient	Std. error	z	P>|z|	95% CI
const	-2.7150	0.193	-14.094	0.000	[-3.093, -2.337]
statistics.average rating	-0.0133	0.023	-0.578	0.563	[-0.059, 0.032]
statistics.sale price	-0.0987	0.008	-11.892	0.000	[-0.115, -0.082]
statistics.total reviews	0.0031	9.09e-05	33.844	0.000	[0.003, 0.003]
genre_comics	-0.8096	0.594	-1.362	0.173	[-1.975, 0.355]
genre_fiction	2.7225	0.151	18.076	0.000	[2.427, 3.018]
genre_genre fiction	2.5190	0.125	20.094	0.000	[2.273, 2.765]
genre_nonfiction	0.7910	0.131	6.036	0.000	[0.534, 1.048]
publisher.type_big five	-0.4993	0.124	-4.022	0.000	[-0.743, -0.256]
publisher.type_indie	-0.4225	0.122	-3.472	0.001	[-0.661, -0.184]
publisher.type_single author	-1.1163	0.132	-8.475	0.000	[-1.374, -0.858]
publisher.type_small/medium	-1.3624	0.125	-10.924	0.000	[-1.607, -1.118]

In the screening regression, total reviews is positively associated with bestseller status (coefficient 0.0031, $p < 0.001$) and sale price is negatively associated (-0.0987, $p < 0.001$). Average rating is not statistically significant in this model ($p = 0.563$). Genre coefficients for fiction and genre fiction are positive and statistically significant relative to the omitted baseline category. These are conditional associations in this fitted specification, not causal effects.

6.1 Features retained for the scratch model

Selected feature	Reason reflected by the workflow
genre	Categorical EDA shows differences in bestseller percentages across genre groups.
publisher.type	Categorical EDA shows differences across publisher-type groups.
statistics.sale price	Sale price has a negative association in the screening Logit output.
statistics.total reviews	Review count has a positive association and is strongly significant in the screening Logit output.

The notebook prints the selected feature list as genre, publisher.type, statistics.sale price and statistics.total reviews. Sales rank is excluded because it directly defines the target; average rating is omitted from the final scratch model.

7. Model preparation and training

Categorical predictors are converted into dummy variables using pandas get_dummies with drop_first=True, then the data are split with stratification. The notebook reports 21,621 training records and 5,406 testing records. The numeric/dummy features are scaled using StandardScaler. A custom LogisticRegressionScratch class implements the sigmoid function, gradient descent and binary cross-entropy/log-loss; an improved model is then trained and evaluated.

Partition	Records
Training	21,621
Testing	5,406
Total	27,027

The train/test split is stratified so that the minority class remains represented in both sets. Standardisation can support gradient-based optimisation when predictor scales differ. The custom implementation is Python/NumPy code, rather than PySpark ML code; this report preserves that distinction.

8. Initial and improved model results

Model	Accuracy	Precision	Recall	F1-score
Initial model (threshold 0.50)	0.8827	0.8435	0.1360	0.2343

Improved model (threshold 0.50)	0.8873	0.7549	0.2160	0.3359

The improved model raises recall from 0.1360 to 0.2160 and F1-score from 0.2343 to 0.3359, while accuracy increases from 0.8827 to 0.8873. Precision decreases from 0.8435 to 0.7549, indicating that the improved model identifies more true bestsellers but also makes more positive predictions that are false positives. These values are the notebook's printed results at threshold 0.50.

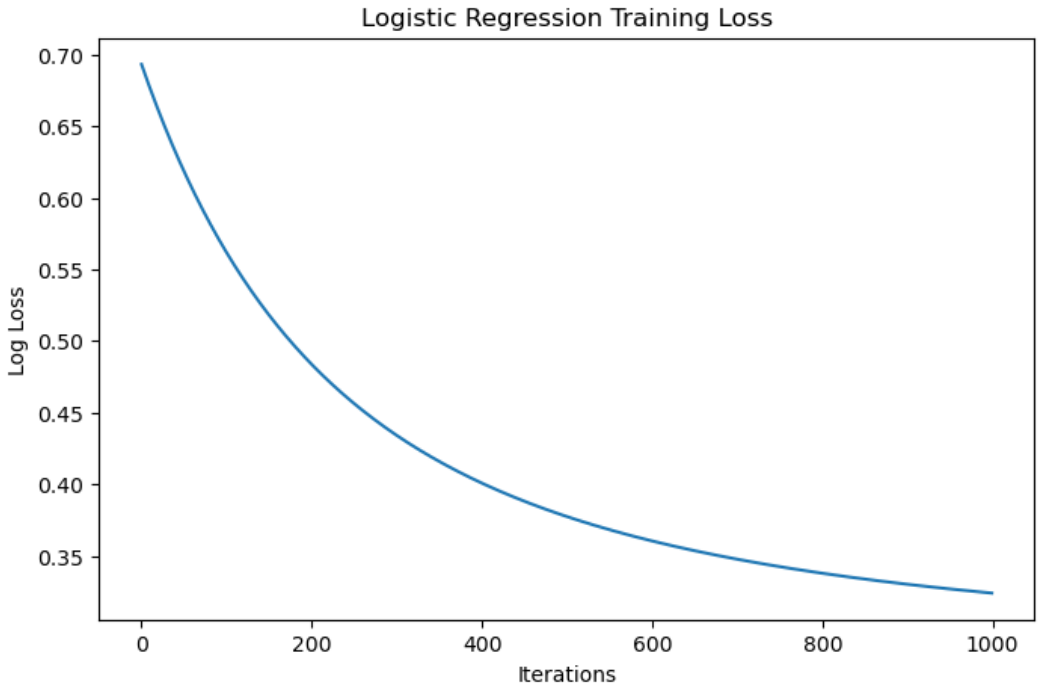


Figure 7. Training loss for the initial scratch logistic-regression model.

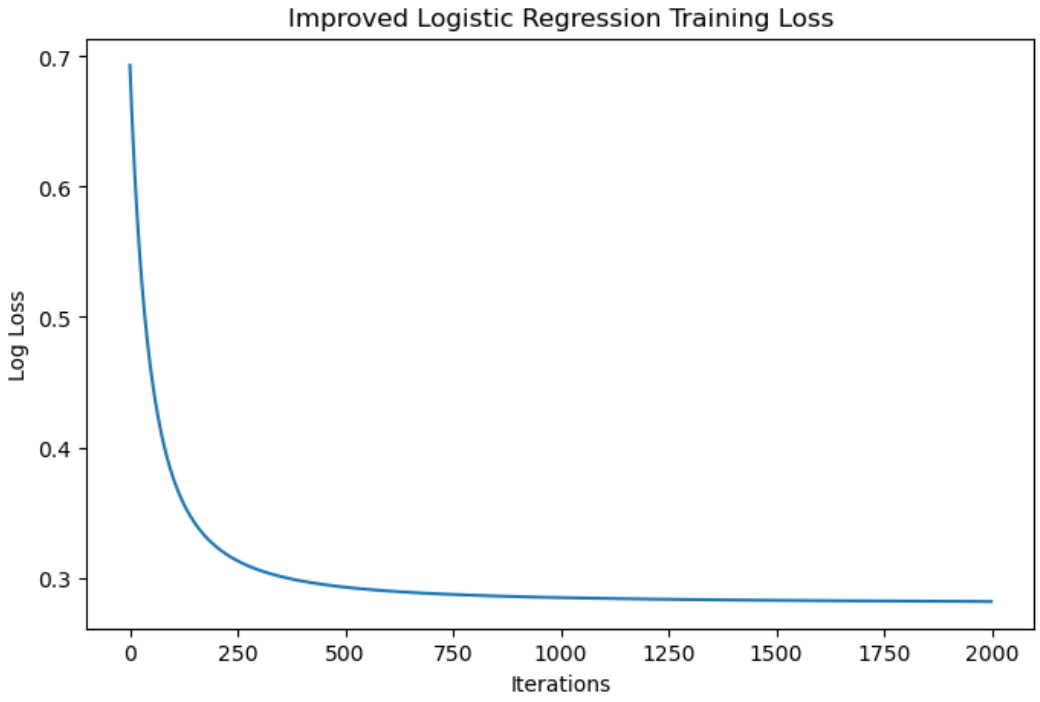


Figure 8. Training loss for the improved scratch logistic-regression model.

9. Threshold optimisation

The notebook evaluates thresholds of 0.30, 0.40, 0.50, 0.60 and 0.70 using predicted probabilities from the improved model. It selects the threshold with the highest F1-score rather than the threshold with the highest accuracy.

Threshold	Accuracy	Precision	Recall	F1-score
0.30	0.8663	0.4934	0.5217	0.5072
0.40	0.8896	0.6812	0.3058	0.4221
0.50	0.8873	0.7549	0.2160	0.3359
0.60	0.8846	0.7834	0.1725	0.2828
0.70	0.8835	0.8487	0.1417	0.2428

Exact notebook selection: Best threshold = 0.3; Best F1-score = 0.5072.

Threshold 0.30 produces the highest recall (0.5217) and F1-score (0.5072) among the tested thresholds. Threshold 0.40 has the highest accuracy (0.8896), but its recall and F1-score are lower. As the threshold rises, precision generally rises while recall falls. The selected threshold is appropriate if the objective prioritises identifying more bestseller books over maximising overall accuracy.

10. Final evaluation and confusion matrix

Metric	Final result at threshold 0.30
Accuracy	0.8663 (86.63%)
Precision	0.4934 (49.34%)
Recall	0.5217 (52.17%)
F1-score	0.5072 (50.72%)

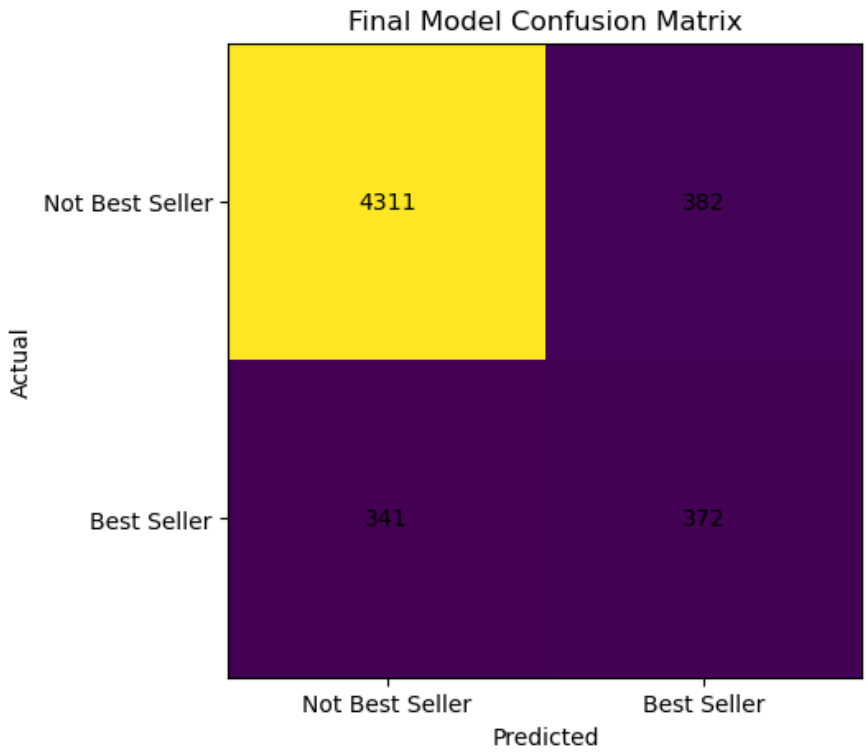


Figure 9. Final confusion matrix at the selected threshold of 0.30.

Actual class	Predicted not bestseller (0)	Predicted bestseller (1)
Not bestseller (0)	4,311 true negatives	382 false positives
Bestseller (1)	341 false negatives	372 true positives

The confusion matrix is [[4311, 382], [341, 372]] in the notebook's class order. The model correctly identifies 372 of 713 positive test cases and misses 341; it also predicts 382 non-bestsellers as bestsellers. Recall is 372 / (372 + 341) = 0.5217, and precision is 372 / (372 + 382) = 0.4934. Thus, fewer than half of the books predicted as bestsellers are true positives in this test set, while the selected threshold identifies just over half of actual bestsellers.

11. Feature weights

The notebook prints the improved model's weights sorted by absolute magnitude. Because the inputs include standardised numeric values and dummy-coded categorical values, weights describe contribution to the model's linear score under this encoding. They should not be interpreted as causal importance or compared naively across differently encoded/scaled predictors.

Feature	Weight	Absolute weight
statistics.total reviews	1.295380	1.295380
genre_genre fiction	0.841128	0.841128
statistics.sale price	-0.405648	0.405648
publisher.type_small/medium	-0.394486	0.394486
genre_fiction	0.330907	0.330907
publisher.type_single author	-0.175831	0.175831
genre_comics	-0.150861	0.150861
genre_foreign language	-0.105249	0.105249
publisher.type_indie	0.075633	0.075633
genre_nonfiction	0.005450	0.005450
publisher.type_big five	-0.000454	0.000454

Total reviews have the largest absolute weight (1.295380), followed by the genre_genre fiction indicator (0.841128). Sale price and publisher.type_small/medium have negative weights of −0.405648 and −0.394486. The genre and publisher coefficients are relative to the reference categories created by drop_first=True.

12. Discussion

The notebook's results indicate that bestseller status is associated with genre, publisher type, sale price and review volume. The EDA shows much higher average units sold, revenue and review counts among bestseller-labelled books, and the group-level rates vary substantially by genre and publisher type. The screening regression supports retaining sale price and total reviews, while the scratch model uses the selected four-feature set.

The final threshold decision illustrates a trade-off. At 0.30, F1-score rises to 0.5072 and recall to 0.5217, compared with F1-score 0.3359 and recalls 0.2160 at threshold 0.50. However, precision falls to 0.4934 and accuracy to 0.8663. The threshold is therefore a decision choice rather than a universally superior setting: the appropriate operating point depends on the relative cost of missed bestsellers and false positive alerts.

13. Limitations and recommendations

- Class imbalance: only 13.19% of the dataset is labelled bestseller, so positive-class metrics and class-aware validation are important.
- Predictor availability: the analysis uses fields in the dataset; performance may change if the model is applied to new books where review counts and sales measures are unavailable or measured differently.
- Potential temporal leakage: the dataset's measurement timing is not established in the notebook. Review counts and sales variables may reflect outcomes that occur after publication; a deployment-ready model should verify that every predictor is available at prediction time.
- Evaluation design: the notebook reports one stratified train/test split. Repeated cross-validation or a time-based holdout, if dates are available, would provide stronger evidence about generalisation.
- Model comparison: compare the scratch model with a well-validated baseline and regularised logistic regression, and consider class weighting or resampling only within training folds.
- Threshold choice: choose the threshold based on business costs and validation data, not the test set, if the goal is a production system.
- Spark scalability: the notebook uses Spark for ingestion and a null audit but moves to pandas for EDA and modelling. For larger datasets, perform aggregations and feature preparation in Spark and use Spark ML pipelines where appropriate.

14. Conclusion

The executed notebook analyses 27,027 book records and creates a bestseller target from sales rank ≤ 5,000. The target is imbalanced, with 3,566 bestsellers (13.19%). EDA shows higher average commercial activity and review volume for bestseller-labelled books, while genre and publisher type exhibit distinct bestseller rates. Feature screening identifies meaningful associations for sale price, review count and several categorical indicators.

The improved scratch logistic-regression model's default-threshold results are accuracy 0.8873, precision 0.7549, recall 0.2160 and F1-score 0.3359. Selecting the tested threshold 0.30 by F1-score yields accuracy 0.8663, precision 0.4934, recall 0.5217 and F1-score 0.5072, with 372 true positives, 341 false negatives, 382 false positives and 4,311 true negatives. These exact values are used throughout this report to remain consistent with the executed code.

References

1. AssessmentPart2.ipynb (2026) Book Sales Bestseller Classification: executed Jupyter notebook, including source code, printed outputs and embedded figures. Unpublished assessment material, supplied by the user.

2. Damji, J.S., Wenig, B., Das, T. and Lee, D. (2020) Learning Spark: Lightning-Fast Data Analytics. 2nd edn. Sebastopol, CA: O'Reilly Media.
3. Feng, C., Wang, H., Lu, N., Chen, T., He, H. and Lu, Y. (2014) 'Logistic regression and its diagnostics: an overview', Shanghai Archives of Psychiatry, 26(2), pp. 105–117.
4. James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J. (2023) An Introduction to Statistical Learning: with Applications in Python. New York: Springer.
5. Sokolova, M. and Lapalme, G. (2009) 'A systematic analysis of performance measures for classification tasks', Information Processing & Management, 45(4), pp. 427–437.

Appendix A. Jupiter Notebook

In [1]:
```
# Book Sales Bestseller Classification Using PySpark
## Exploratory Data Analysis, Feature Selection and Logistic Regression
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.max_columns", None)

df = pd.read_csv("publishers.csv")
print("Shape:", df.shape)
df.head()
```

```
Shape: (27027, 13)
```

Out[1]:

	genre	sold by	daily average.amazon revenue	daily average.author revenue	daily average.gross sales	daily average.publisher revenue
0	genre fiction	HarperCollins Publishers	6832.0	6832.0	34160.0	20496.0
1	genre fiction	HarperCollins Publishers	2487.5	2487.5	12437.5	7462.5
2	genre fiction	Amazon Digital Services, Inc.	9559.0	9559.0	47795.0	28677.0
3	fiction	Hachette Book Group	8250.0	8250.0	41250.0	24750.0
4	genre fiction	Penguin Group (USA) LLC	7590.5	7590.5	37952.5	22771.5

In [2]:
```
#Structure and data quality
print("Dataset shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())
```

```
Dataset shape: (27027, 13)

Columns:
['genre', 'sold by', 'daily average.amazon revenue', 'daily average.author revenue',
'daily average.gross sales', 'daily average.publisher revenue', 'daily average.units
sold', 'publisher.name', 'publisher.type', 'statistics.average rating', 'statistics.
sale price', 'statistics.sales rank', 'statistics.total reviews']

Data types:
genre                               object
sold by                             object
daily average.amazon revenue       float64
daily average.author revenue       float64
daily average.gross sales          float64
daily average.publisher revenue    float64
daily average.units sold             int64
publisher.name                      object
publisher.type                      object
statistics.average rating          float64
statistics.sale price              float64
statistics.sales rank                int64
statistics.total reviews             int64
dtype: object

Missing values:
genre                              0
sold by                            0
daily average.amazon revenue       0
daily average.author revenue       0
daily average.gross sales          0
daily average.publisher revenue    0
daily average.units sold           0
publisher.name                     0
publisher.type                     0
statistics.average rating          0
statistics.sale price              0
statistics.sales rank              0
statistics.total reviews           0
dtype: int64

Duplicate rows: 0
```

In [3]:
```
## Descriptive statistics
df.describe()
```

Out[3]:

	daily average.amazon revenue	daily average.author revenue	daily average.gross sales	daily average.publisher revenue	daily average.units sold	st
count	27027.000000	27027.000000	27027.000000	27027.000000	27027.000000	
mean	37.978937	41.465654	150.572864	71.128273	29.614053	
std	250.995881	223.321357	865.653698	481.337817	157.739500	
min	0.004000	0.004000	0.020000	0.000000	1.000000	
25%	3.992000	4.036000	16.000000	1.794000	4.000000	
50%	9.576000	10.152000	41.860000	16.776000	9.000000	
75%	21.978000	25.116000	98.910000	47.952000	14.000000	
max	12974.000000	9559.000000	47795.000000	28677.000000	7000.000000	

In [4]:

```
## PySpark setup and column-name cleaning
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, count, sum as spark_sum, when, round as spark_round,
    avg, min as spark_min, max as spark_max
)

spark = (
    SparkSession.builder
    .appName("AmazonBookAnalysis")
    .getOrCreate()
)

spark_df = spark.read.csv(
    "publishers.csv",
    header=True,
    inferSchema=True
)

for old_name in spark_df.columns:
    new_name = old_name.lower().replace(" ", "_").replace(".", "_")
    spark_df = spark_df.withColumnRenamed(old_name, new_name)

spark_df.printSchema()
spark_df.show(5, truncate=False)
```

```
root
 |-- genre: string (nullable = true)
 |-- sold_by: string (nullable = true)
 |-- daily_average_amazon_revenue: double (nullable = true)
 |-- daily_average_author_revenue: double (nullable = true)
 |-- daily_average_gross_sales: double (nullable = true)
 |-- daily_average_publisher_revenue: double (nullable = true)
 |-- daily_average_units_sold: integer (nullable = true)
 |-- publisher_name: string (nullable = true)
 |-- publisher_type: string (nullable = true)
 |-- statistics_average_rating: double (nullable = true)
 |-- statistics_sale_price: double (nullable = true)
 |-- statistics_sales_rank: integer (nullable = true)
 |-- statistics_total_reviews: integer (nullable = true)

+-------------+------------------------------+----------------------------+---------
------------------+-------------------------+--------------------------------+------
------------------+-------------------------+-------------+----------------------
---+---------------------+--------------------+-----------------------+
|genre        |sold_by                       |daily_average_amazon_revenue|daily_ave
rage_author_revenue|daily_average_gross_sales|daily_average_publisher_revenue|daily_
average_units_sold|publisher_name             |publisher_type|statistics_average_rat
ing|statistics_sale_price|statistics_sales_rank|statistics_total_reviews|
+-------------+------------------------------+----------------------------+---------
------------------+-------------------------+--------------------------------+------
------------------+-------------------------+-------------+----------------------
---+---------------------+--------------------+-----------------------+
|genre fiction|HarperCollins Publishers      |6832.0                      |6832.0
|34160.0                  |20496.0                         |7000                    |
Katherine Tegen Books      |big five      |4.57                    |4.88
|1                    |9604                     |
|genre fiction|HarperCollins Publishers      |2487.5                      |2487.5
|12437.5                  |7462.5                          |6250                    |
HarperCollins e-books      |big five      |4.47                    |1.99
|2                    |450                      |
|genre fiction|Amazon Digital Services,  Inc.|9559.0                      |9559.0
|47795.0                  |28677.0                         |5500                    |
(Small or Medium Publisher)|small/medium  |4.16                    |8.69
|3                    |30                       |
|fiction      |Hachette Book Group           |8250.0                      |8250.0
|41250.0                  |24750.0                         |5500                    |
Little, Brown and Company  |big five      |3.84                    |7.5
|3                    |3747                     |
|genre fiction|Penguin Group (USA) LLC       |7590.5                      |7590.5
|37952.5                  |22771.5                         |4750                    |
Dutton Children's          |big five      |4.75                    |7.99
|4                    |9174                     |
+-------------+------------------------------+----------------------------+---------
------------------+-------------------------+--------------------------------+------
------------------+-------------------------+-------------+----------------------
---+---------------------+--------------------+-----------------------+
only showing top 5 rows
```

In [5]:
```
#Spark missing values
missing_exprs = [
    spark_sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
```

```
    for c in spark_df.columns
]

spark_df.select(*missing_exprs).show()
```

```
+-----+-------+--------------------------+--------------------------+----------
--------------+-------------------------------+----------------------+----------
--+--------------+------------------------+--------------------+----------------
---+------------------------+
|genre|sold_by|daily_average_amazon_revenue|daily_average_author_revenue|daily_avera
ge_gross_sales|daily_average_publisher_revenue|daily_average_units_sold|publisher_na
me|publisher_type|statistics_average_rating|statistics_sale_price|statistics_sales_r
ank|statistics_total_reviews|
+-----+-------+--------------------------+--------------------------+----------
--------------+-------------------------------+----------------------+----------
--+--------------+------------------------+--------------------+----------------
---+------------------------+
|    0|      0|                         0|                         0|
0|                             0|                      0|            0|
0|                        0|                   0|                    0|
0|
+-----+-------+--------------------------+--------------------------+----------
--------------+-------------------------------+----------------------+----------
--+--------------+------------------------+--------------------+----------------
---+------------------------+
```

In [6]:
```
## Bestseller target

#A book is labelled as a bestseller when its Amazon sales rank is 5,000 or lower.
#Sales rank itself is not used as a predictor because it directly defines the targe
rank = df["statistics.sales rank"]

thresholds = [100, 500, 1000, 5000, 10000]

for threshold in thresholds:
    n = (rank <= threshold).sum()
    pct = n / len(df) * 100
    print(f"Rank <= {threshold}: {n} books ({pct:.2f}%)")

df["BestSeller"] = (df["statistics.sales rank"] <= 5000).astype(int)

print(df["BestSeller"].value_counts())
print((df["BestSeller"].value_counts(normalize=True) * 100).round(2))
```

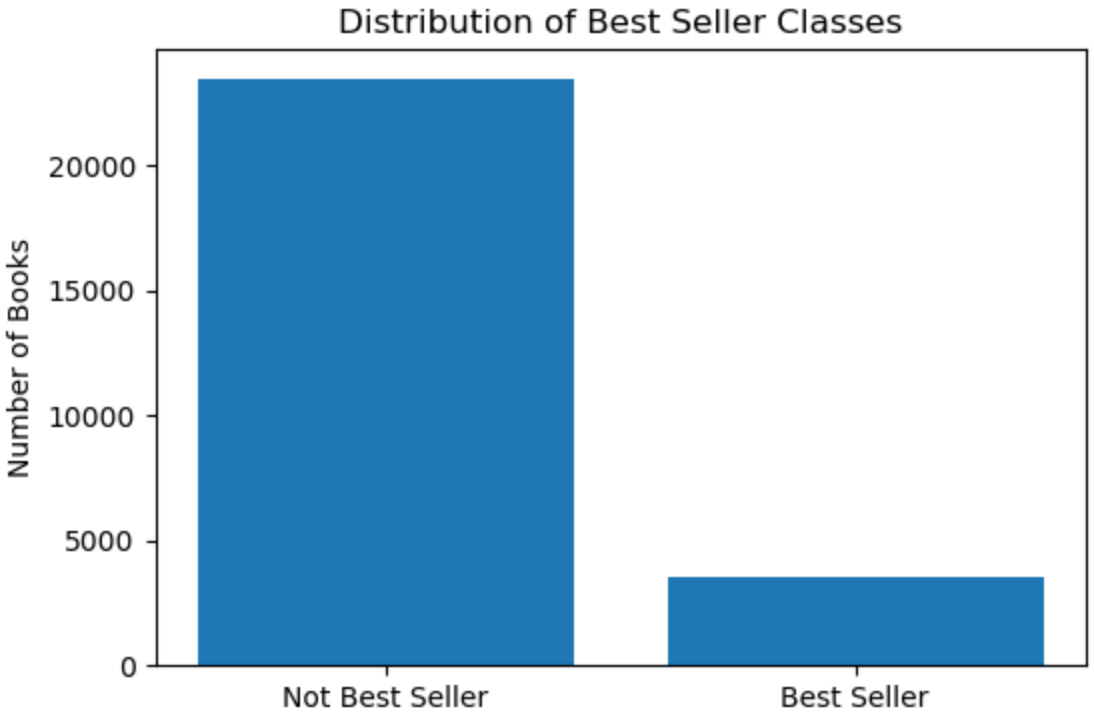
```
Rank <= 100: 99 books (0.37%)
Rank <= 500: 453 books (1.68%)
Rank <= 1000: 854 books (3.16%)
Rank <= 5000: 3566 books (13.19%)
Rank <= 10000: 6014 books (22.25%)
BestSeller
0    23461
1     3566
Name: count, dtype: int64
BestSeller
0    86.81
1    13.19
Name: proportion, dtype: float64
```

In [7]:
```
## Distribution of Best Seller Classes

#The target is expected to be imbalanced, so accuracy will not be used as the only
plt.figure(figsize=(6,4))

counts = df["BestSeller"].value_counts().sort_index()
plt.bar(["Not Best Seller", "Best Seller"], counts.values)

plt.title("Distribution of Best Seller Classes")
plt.ylabel("Number of Books")
plt.show()
```

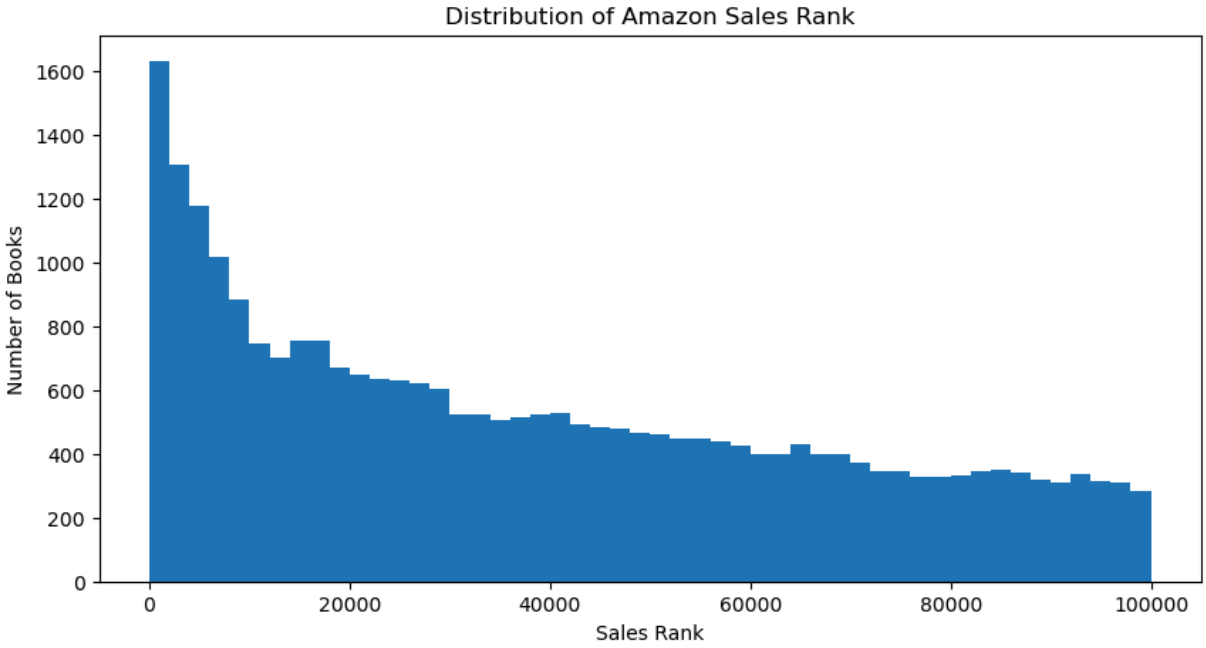


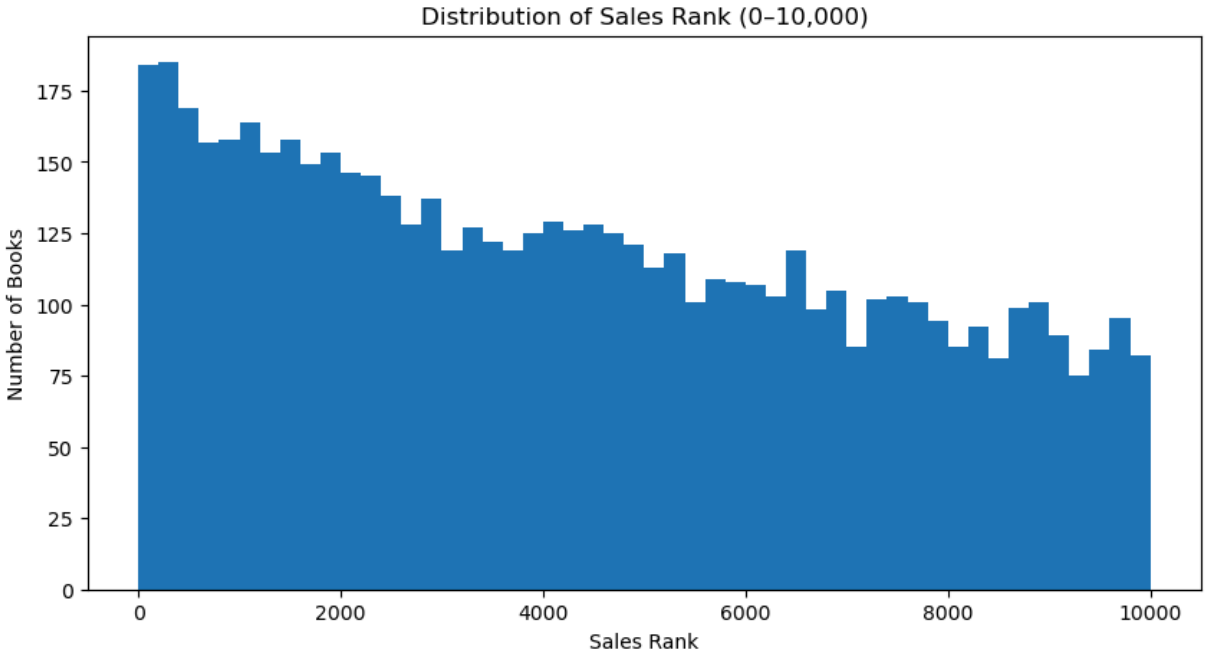
In [8]:
```
## Distribution of Amazon Sales Rank

plt.figure(figsize=(10,5))
plt.hist(df["statistics.sales rank"], bins=50)
plt.title("Distribution of Amazon Sales Rank")
plt.xlabel("Sales Rank")
plt.ylabel("Number of Books")
```

```
plt.show()

plt.figure(figsize=(10,5))
plt.hist(df["statistics.sales rank"], bins=50, range=(0,10000))
plt.title("Distribution of Sales Rank (0–10,000)")
plt.xlabel("Sales Rank")
plt.ylabel("Number of Books")
plt.show()
```





In [9]:
```
## Correlation Matrix of Numerical Variables

numerical_columns = [
    "daily average.amazon revenue",
    "daily average.author revenue",
    "daily average.gross sales",
    "daily average.publisher revenue",
```

```
    "daily average.units sold",
    "statistics.average rating",
    "statistics.sale price",
    "statistics.sales rank",
    "statistics.total reviews",
    "BestSeller"
]

plt.figure(figsize=(12,8))
corr = df[numerical_columns].corr()

sns.heatmap(corr, annot=True, fmt=".2f")
plt.title("Correlation Matrix of Numerical Variables")
plt.show()

print(
    df[
        numerical_columns
    ].corr()["BestSeller"].sort_values(ascending=False)
)
```



```
BestSeller                          1.000000
daily average.units sold            0.348056
daily average.author revenue        0.323436
statistics.total reviews            0.293715
daily average.gross sales           0.293240
daily average.amazon revenue        0.270493
daily average.publisher revenue     0.236261
statistics.average rating           0.007968
statistics.sale price              -0.108989
statistics.sales rank              -0.481442
Name: BestSeller, dtype: float64
```

In [10]:
```
## Comparison of Best Sellers and Non-Best Sellers

numerical_features = [
    "daily average.amazon revenue",
    "daily average.author revenue",
    "daily average.gross sales",
    "daily average.publisher revenue",
    "daily average.units sold",
    "statistics.average rating",
    "statistics.sale price",
    "statistics.total reviews"
]

comparison = df.groupby("BestSeller")[numerical_features].mean().T
comparison.columns = ["Not Best Seller", "Best Seller"]
print(comparison)
```

```
                                Not Best Seller  Best Seller
daily average.amazon revenue          11.510298   212.118233
daily average.author revenue          13.305937   226.730686
daily average.gross sales             51.609108   801.663348
daily average.publisher revenue       26.792873   362.814429
daily average.units sold               8.209880   170.433819
statistics.average rating              4.156693     4.180137
statistics.sale price                  7.048089     4.997182
statistics.total reviews              82.400580   489.387830
```

In [11]:
```
## Bestseller distribution by genre and publisher type

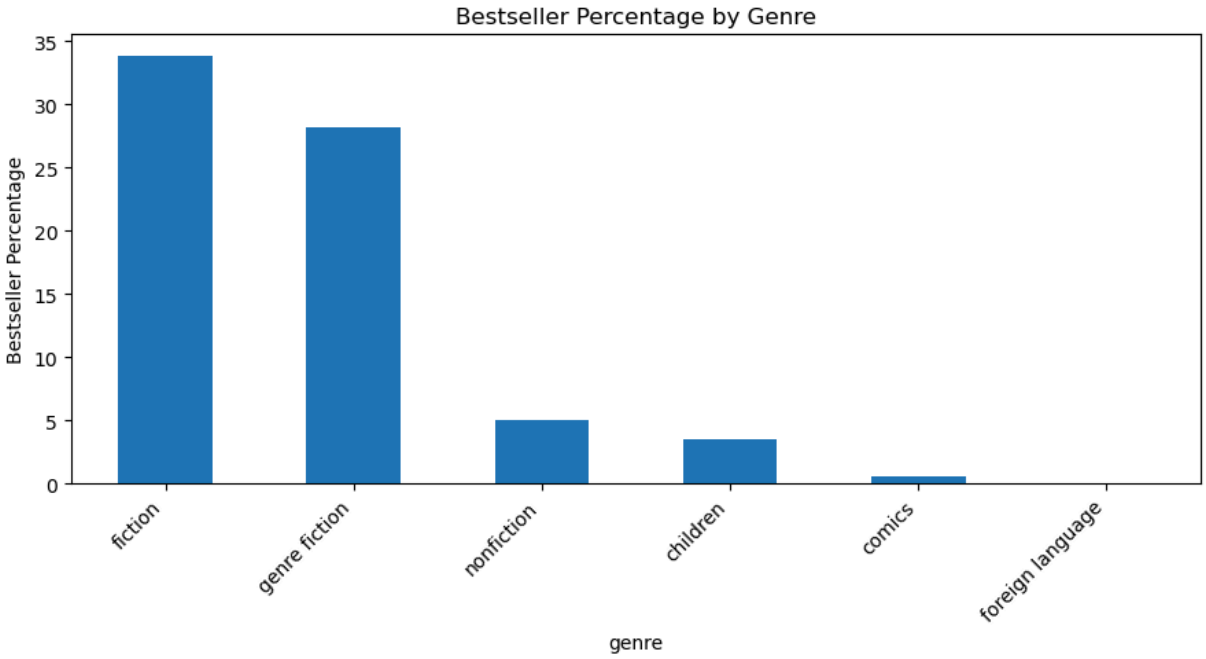
genre_analysis = (
    df.groupby("genre")["BestSeller"]
      .agg(total_books="count", bestsellers="sum")
      .assign(
          bestseller_percentage=lambda x:
          (x["bestsellers"] / x["total_books"] * 100).round(2)
      )
      .sort_values("bestseller_percentage", ascending=False)
)

print(genre_analysis)

genre_analysis["bestseller_percentage"].plot(kind="bar", figsize=(9,5))
plt.title("Bestseller Percentage by Genre")
plt.ylabel("Bestseller Percentage")
```

```
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

genre	total_books	bestsellers	bestseller_percentage
fiction	733	248	33.83
genre fiction	8903	2512	28.22
nonfiction	14161	715	5.05
children	2541	88	3.46
comics	568	3	0.53
foreign language	121	0	0.00

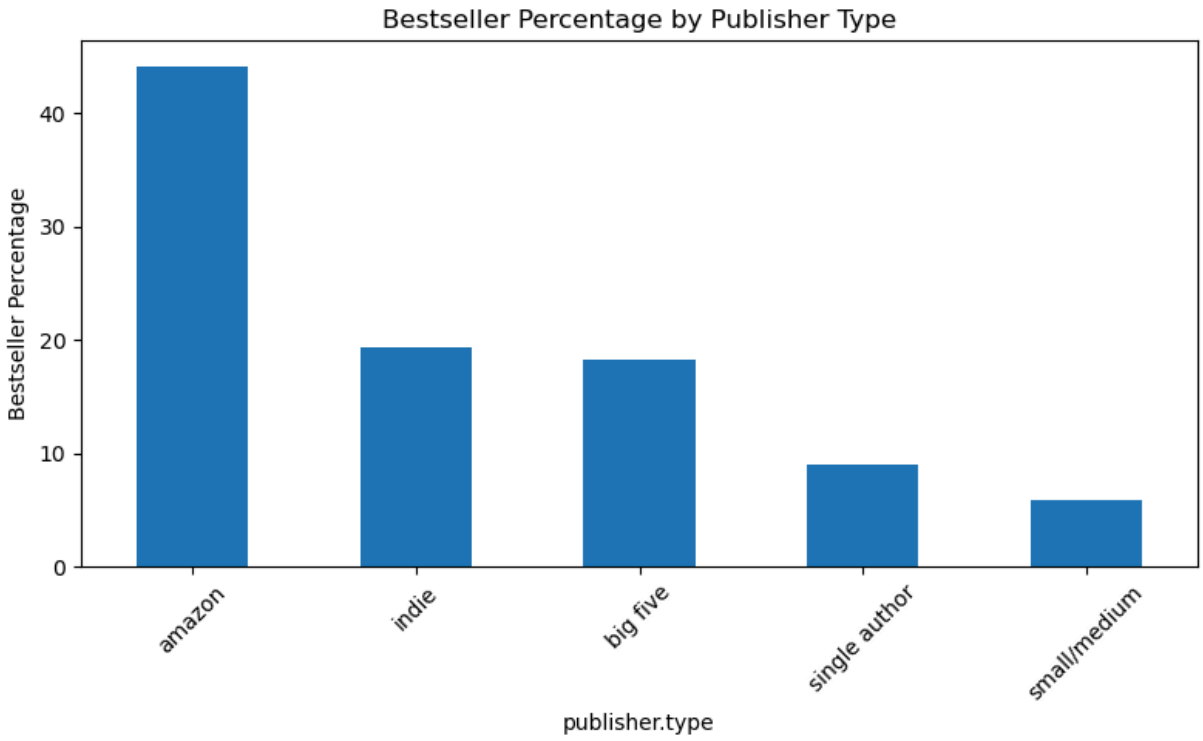


In [12]:
```
#Publisher-type analysis
publisher_analysis = (
    df.groupby("publisher.type")["BestSeller"]
      .agg(total_books="count", bestsellers="sum")
      .assign(
          bestseller_percentage=lambda x:
          (x["bestsellers"] / x["total_books"] * 100).round(2)
      )
      .sort_values("bestseller_percentage", ascending=False)
)

print(publisher_analysis)

publisher_analysis["bestseller_percentage"].plot(kind="bar", figsize=(8,5))
plt.title("Bestseller Percentage by Publisher Type")
plt.ylabel("Bestseller Percentage")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

publisher.type	total_books	bestsellers	bestseller_percentage
amazon	423	187	44.21
indie	5946	1147	19.29
big five	7309	1333	18.24
single author	3608	323	8.95
small/medium	9741	576	5.91



In [13]:
```
## Feature selection using p-values

#Candidate variables are genre, publisher type, average rating, sale price and tota
#Sales rank is excluded because it defines the target.

import statsmodels.api as sm

df_selection = df.copy()

df_selection["genre"] = df_selection["genre"].replace(
    "foreign language", "other"
)

sold_counts = df_selection["sold by"].value_counts()
rare_sellers = sold_counts[sold_counts < 10].index

df_selection["sold by"] = df_selection["sold by"].replace(
    rare_sellers, "Other"
)

df_pvalue = df_selection[
    df_selection["genre"] != "other"
].copy()

features_pvalue = [
    "genre",
```

```
    "publisher.type",
    "statistics.average rating",
    "statistics.sale price",
    "statistics.total reviews"
]

X_pvalue = pd.get_dummies(
    df_pvalue[features_pvalue],
    columns=["genre", "publisher.type"],
    drop_first=True
).astype(float)

X_pvalue = sm.add_constant(X_pvalue)
y_pvalue = df_pvalue["BestSeller"]

logit_model = sm.Logit(y_pvalue, X_pvalue)
result = logit_model.fit(maxiter=200, disp=True)

print(result.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.280591
         Iterations 9
                           Logit Regression Results
==============================================================================
Dep. Variable:             BestSeller   No. Observations:                26906
Model:                          Logit   Df Residuals:                    26894
Method:                           MLE   Df Model:                           11
Date:                Wed, 21 Oct 2026   Pseudo R-squ.:                 0.2827
Time:                        12:00:31   Log-Likelihood:                -7549.6
converged:                       True   LL-Null:                       -10525.
Covariance Type:            nonrobust   LLR p-value:                     0.000
==============================================================================
============
                                 coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
------------
const                          -2.7150      0.193    -14.094      0.000      -3.093
-2.337
statistics.average rating      -0.0133      0.023     -0.578      0.563      -0.059
0.032
statistics.sale price          -0.0987      0.008    -11.892      0.000      -0.115
-0.082
statistics.total reviews        0.0031   9.09e-05     33.844      0.000       0.003
0.003
genre_comics                   -0.8096      0.594     -1.362      0.173      -1.975
0.355
genre_fiction                   2.7225      0.151     18.076      0.000       2.427
3.018
genre_genre fiction             2.5190      0.125     20.094      0.000       2.273
2.765
genre_nonfiction                0.7910      0.131      6.036      0.000       0.534
1.048
publisher.type_big five        -0.4993      0.124     -4.022      0.000      -0.743
-0.256
publisher.type_indie           -0.4225      0.122     -3.472      0.001      -0.661
-0.184
publisher.type_single author   -1.1163      0.132     -8.475      0.000      -1.374
-0.858
publisher.type_small/medium    -1.3624      0.125    -10.924      0.000      -1.607
-1.118
==============================================================================
============
```

In [14]:

```python
#Final selected features
selected_features = [
    "genre",
    "publisher.type",
    "statistics.sale price",
    "statistics.total reviews"
]

print("Selected features:")
for feature in selected_features:
    print("-", feature)
```

```
Selected features:
- genre
- publisher.type
- statistics.sale price
- statistics.total reviews
```

In [15]:
```python
## Prepare modelling data

#Categorical predictors are converted into dummy variables. The data are split usin
#so that the minority class remains represented in both training and testing sets.

#Train/test split and scaling
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = pd.get_dummies(
    df[selected_features],
    columns=["genre", "publisher.type"],
    drop_first=True
).astype(float)

y = df["BestSeller"].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

```
Training records: 21621
Testing records: 5406
```

In [16]:
```python
## Logistic regression from scratch

#The model uses the sigmoid function, gradient descent and binary cross-entropy/log

class LogisticRegressionScratch:

    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.weights = None
        self.bias = 0
        self.loss_history = []

    def sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))
```

```
    def fit(self, X, y):
        n_samples, n_features = X.shape

        self.weights = np.zeros(n_features)
        self.bias = 0
        self.loss_history = []

        for _ in range(self.n_iterations):
            linear_model = np.dot(X, self.weights) + self.bias
            predictions = self.sigmoid(linear_model)

            dw = (1 / n_samples) * np.dot(
                X.T, predictions - y
            )
            db = (1 / n_samples) * np.sum(
                predictions - y
            )

            self.weights -= self.learning_rate * dw
            self.bias -= self.learning_rate * db

            eps = 1e-15
            loss = -(1 / n_samples) * np.sum(
                y * np.log(predictions + eps) +
                (1 - y) * np.log(1 - predictions + eps)
            )

            self.loss_history.append(loss)

    def predict_probability(self, X):
        linear_model = np.dot(X, self.weights) + self.bias
        return self.sigmoid(linear_model)

    def predict(self, X, threshold=0.5):
        probabilities = self.predict_probability(X)
        return (probabilities >= threshold).astype(int)
```

In [17]:

```
#Initial and improved models
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
)

model = LogisticRegressionScratch(
    learning_rate=0.01,
    n_iterations=1000
)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled, threshold=0.5)

print("Initial model")
print("Accuracy :", round(float(accuracy_score(y_test, y_pred)), 4))
print("Precision:", round(float(precision_score(y_test, y_pred, zero_division=0)),
print("Recall   :", round(float(recall_score(y_test, y_pred, zero_division=0)), 4))
```

```
print("F1-score :", round(float(f1_score(y_test, y_pred, zero_division=0)), 4))

improved_model = LogisticRegressionScratch(
    learning_rate=0.05,
    n_iterations=2000
)
improved_model.fit(X_train_scaled, y_train)

y_pred_improved = improved_model.predict(
    X_test_scaled, threshold=0.5
)

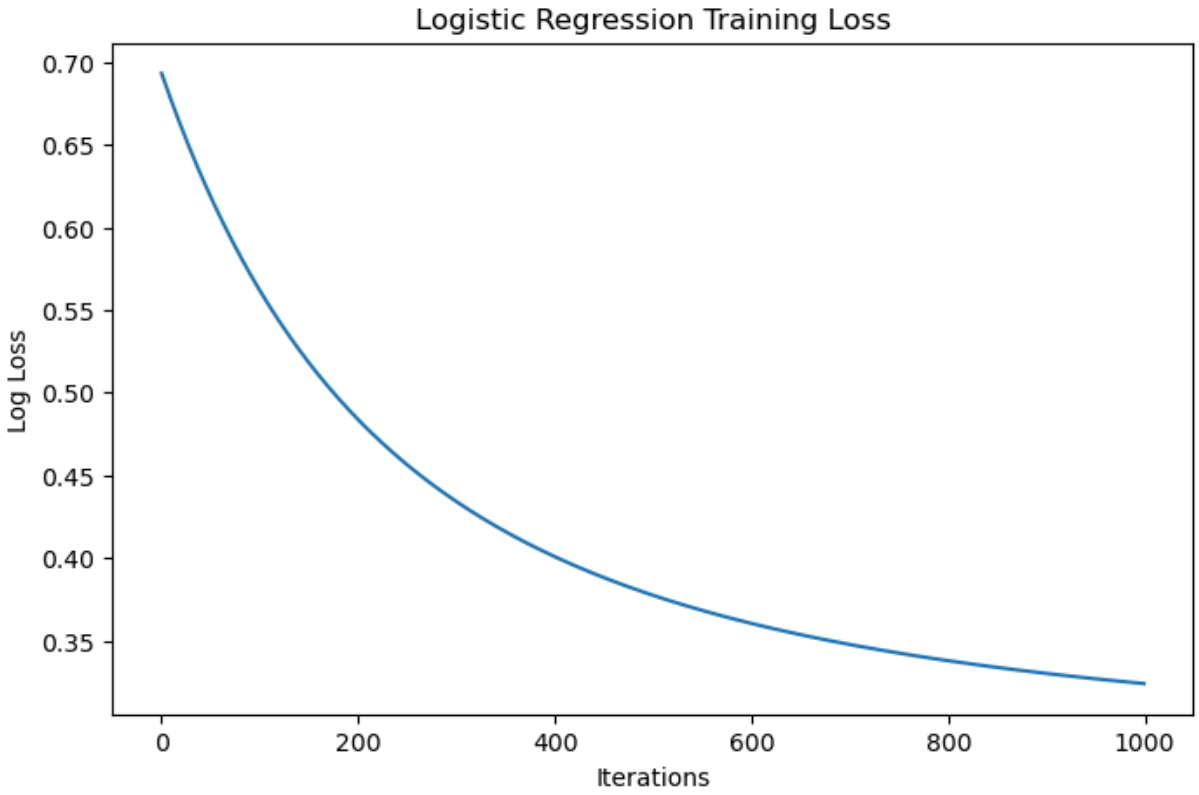
print("\nImproved model")
print("Accuracy :", round(float(accuracy_score(y_test, y_pred_improved)), 4))
print("Precision:", round(float(precision_score(y_test, y_pred_improved, zero_divis
print("Recall   :", round(float(recall_score(y_test, y_pred_improved, zero_division
print("F1-score :", round(float(f1_score(y_test, y_pred_improved, zero_division=0))
```

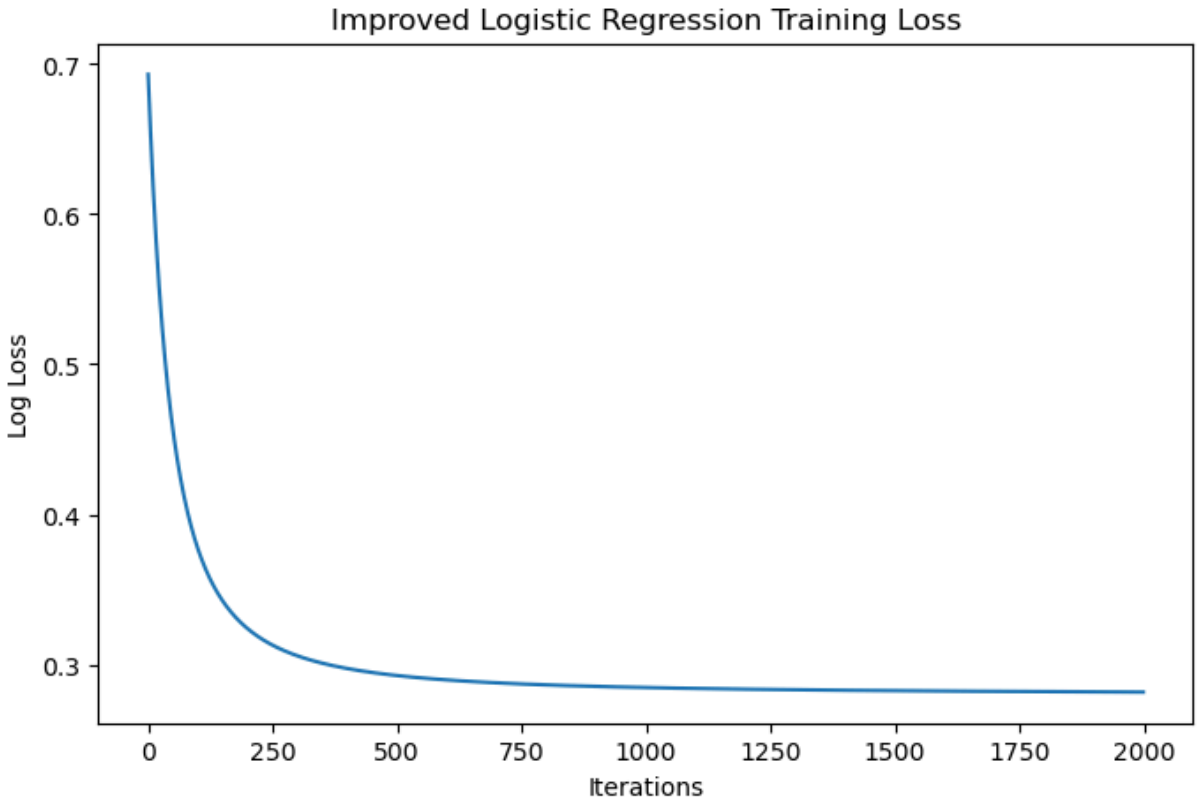
```
Initial model
Accuracy : 0.8827
Precision: 0.8435
Recall   : 0.136
F1-score : 0.2343

Improved model
Accuracy : 0.8873
Precision: 0.7549
Recall   : 0.216
F1-score : 0.3359
```

In [18]:
```
#Training loss
plt.figure(figsize=(8,5))
plt.plot(model.loss_history)
plt.xlabel("Iterations")
plt.ylabel("Log Loss")
plt.title("Logistic Regression Training Loss")
plt.show()

plt.figure(figsize=(8,5))
plt.plot(improved_model.loss_history)
plt.xlabel("Iterations")
plt.ylabel("Log Loss")
plt.title("Improved Logistic Regression Training Loss")
plt.show()
```





In [19]:
```
## Threshold optimisation

#The threshold is selected using the highest F1-score rather than the highest accur

y_probability = improved_model.predict_probability(X_test_scaled)
```

```
thresholds = [0.30, 0.40, 0.50, 0.60, 0.70]
threshold_results = []

for threshold in thresholds:
    pred = (y_probability >= threshold).astype(int)

    threshold_results.append({
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1-score": f1_score(y_test, pred, zero_division=0)
    })

threshold_results = pd.DataFrame(threshold_results)
print(threshold_results.round(4))

best_row = threshold_results.loc[
    threshold_results["F1-score"].idxmax()
]

best_threshold = float(best_row["Threshold"])

print("\nBest threshold:", best_threshold)
print("Best F1-score:", round(float(best_row["F1-score"]), 4))
```

```
   Threshold  Accuracy  Precision  Recall  F1-score
0        0.3    0.8663     0.4934  0.5217    0.5072
1        0.4    0.8896     0.6812  0.3058    0.4221
2        0.5    0.8873     0.7549  0.2160    0.3359
3        0.6    0.8846     0.7834  0.1725    0.2828
4        0.7    0.8835     0.8487  0.1417    0.2428

Best threshold: 0.3
Best F1-score: 0.5072
```

In [20]:

```
#Final evaluation and confusion matrix
y_pred_final = (
    y_probability >= best_threshold
).astype(int)

final_accuracy = accuracy_score(y_test, y_pred_final)
final_precision = precision_score(
    y_test, y_pred_final, zero_division=0
)
final_recall = recall_score(
    y_test, y_pred_final, zero_division=0
)
final_f1 = f1_score(
    y_test, y_pred_final, zero_division=0
)

print("Final Model Evaluation")
print("----------------------")
print("Accuracy :", round(float(final_accuracy), 4))
print("Precision:", round(float(final_precision), 4))
```

```
print("Recall   :", round(float(final_recall), 4))
print("F1-score :", round(float(final_f1), 4))

final_cm = confusion_matrix(y_test, y_pred_final)
print("\nFinal Confusion Matrix:")
print(final_cm)

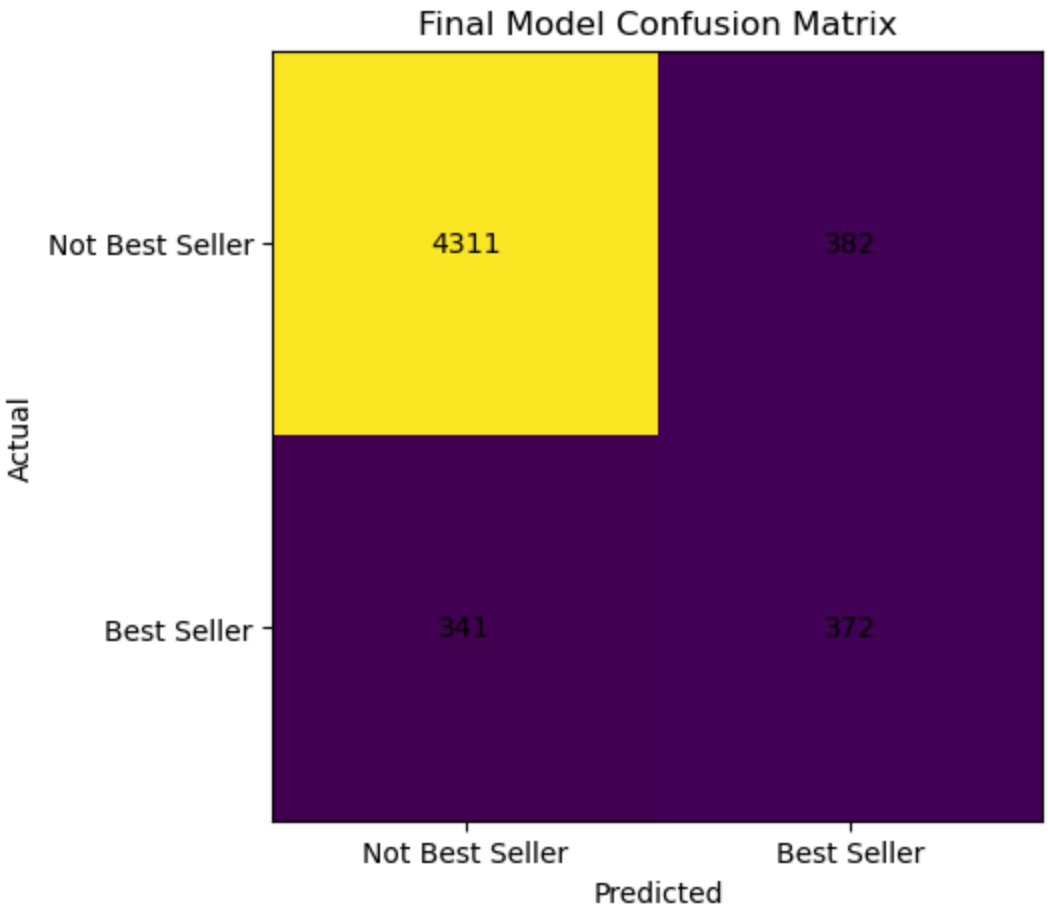
plt.figure(figsize=(6,5))
plt.imshow(final_cm)
plt.title("Final Model Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.xticks([0,1], ["Not Best Seller", "Best Seller"])
plt.yticks([0,1], ["Not Best Seller", "Best Seller"])

for i in range(2):
    for j in range(2):
        plt.text(j, i, final_cm[i,j], ha="center", va="center")

plt.show()
```

```
Final Model Evaluation
----------------------
Accuracy : 0.8663
Precision: 0.4934
Recall   : 0.5217
F1-score : 0.5072

Final Confusion Matrix:
[[4311  382]
 [ 341  372]]
```



In [21]:

```
## Feature weights

#The absolute weight provides a simple indication of which encoded variables have t
#influence on the scratch model's linear score. It should not be interpreted as cau

feature_weights = pd.DataFrame({
    "Feature": X.columns,
    "Weight": improved_model.weights
})

feature_weights["Absolute Weight"] = (
    feature_weights["Weight"].abs()
)

feature_weights = feature_weights.sort_values(
    "Absolute Weight", ascending=False
)

print(feature_weights)
```

	Feature	Weight	Absolute Weight
1	statistics.total reviews	1.295380	1.295380
5	genre_genre fiction	0.841128	0.841128
0	statistics.sale price	-0.405648	0.405648
10	publisher.type_small/medium	-0.394486	0.394486
3	genre_fiction	0.330907	0.330907
9	publisher.type_single author	-0.175831	0.175831
2	genre_comics	-0.150861	0.150861
4	genre_foreign language	-0.105249	0.105249
8	publisher.type_indie	0.075633	0.075633
6	genre_nonfiction	0.005450	0.005450
7	publisher.type_big five	-0.000454	0.000454

In [ ]: